

CAPSLock

Container-Aware Policy Security Lock

Consolidated Scoring & Benchmark Reference

Technical Specification for External Validation

Project ID	25-26J-043
Institution	Sri Lanka Institute of Information Technology
Document Version	1.0
Document Date	April 2026

This document contains proprietary scoring algorithms, benchmark data, and empirical validation results for the four CAPSLock platform components.

Contents

Document Overview.....	4
Components Covered.....	4
How to Read the Benchmark Sections	4
Part I — MEDS (Component 1): Promotion Risk Scorer.....	5
1. Factor Weights	5
2. Per-Factor Scoring Bands	5
3. Environment Risk Thresholds	7
4. Decision Tiers.....	8
5. Key Scenario Results	8
6. Risk Score Calculator — Live Dashboard Captures.....	9
Part II — EAPE (Component 2): Environment-Aware Policy Engine	12
1. Problem Statement.....	12
2. System Architecture	12
3. Policy Application Workflow	13
4. Environment Detection	13
5. Conflict Detection - 5 Dimensions	14
6. Conflict Resolution - 3 Strategies.....	15
7. Policy Conflict Detector - Live Dashboard Captures	15
8. Compliance Validation	19
9. Policy Templates — Environment Tiers	20
10. Advanced Features	20
11. Performance Benchmarks	21
12. Test Coverage.....	21
Part III - SDLB (Component 3): Service Discovery & Load Balancing	23
1. Routing Configuration Thresholds	23
2. Traffic Routing Scenarios - Live Dashboard Capture	25
3. Routing Policy Correctness Benchmark.....	26
4. Health Monitoring & Failover Benchmark.....	29
5. System Overhead Benchmark	30
6. Scalability Benchmark	31
7. Continuous Load Stability Benchmark	33
8. Latency Comparison Across Environments	33
9. SDLB Key Scenario Results Summary	34

Part IV - ICAP Operator (Component 4): Kubernetes-Native Content Scanning	35
1. Overview	35
2. Architecture	35
3. Custom Resource Definition (CRD)	36
5. Adaptive Health Scoring System	37
6. Validated Test Scenarios	40
7. Health Score Scenarios - Live Dashboard Capture	41
8. Prometheus Observability	42
9. Alerting Rules (PrometheusRule)	43
10. SLO Definitions	43
11. RBAC & Security Posture	44
12. Novel Contribution vs. Industry Baseline	44
Part V - Cross-Cutting Compliance Frameworks	45
1. Framework Versions	45
2. PCI-DSS Control Mappings	45
Consolidated Benchmark Results Summary	46
MEDS (Component 1)	46
EAPE (Component 2)	46
SDLB (Component 3)	47
ICAP Operator (Component 4)	48

Document Overview

This report consolidates the scoring specifications, benchmark procedures, and empirical validation results for all four components of the **CAPSLock** Kubernetes security platform. It is intended for external academic and industry reviewers evaluating the technical contribution of each component, and serves as the definitive reference for the scoring models, thresholds, and measured outcomes that underpin the platform's design.

The document is organised by component. Each part combines the authoritative scoring model (weights, bands, thresholds, decision tiers) with its corresponding empirical validation — benchmarks run against a live cluster, test coverage figures, and live dashboard captures from the integrated CAPSLock frontend. A final cross-cutting section summarises the compliance framework mappings applied across the platform.

Components Covered

Part	Component	Responsibility	Primary Contribution
I	MEDS (C1) — Multi-Environment Deployment System	Promotion control	6-factor weighted promotion-risk scorer with environment-gated decision tiers
II	EAPE (C2) — Environment-Aware Policy Engine	Policy selection & compliance	7-factor environment classification + policy-as-code translation (OPA/Kyverno)
III	SDLB (C3) — Service Discovery & Load Balancing	Inspection traffic routing	Traffic-trend hysteresis state machine with ICAP-health-driven spread/single collapse
IV	ICAP Operator (C4) — Kubernetes Content Scanning	Content inspection lifecycle	6-dimension adaptive health scoring with context-aware weight redistribution

How to Read the Benchmark Sections

Scoring models describe how each component converts raw signals (replica counts, latency, version deltas, namespace labels, etc.) into composite numeric scores and discrete decisions.

Scoring bands enumerate the value ranges for every input factor. **Decision tiers** specify the action taken at each score threshold. **Empirical results** report the observed outcomes against those thresholds from controlled tests on a live cluster.

Screenshots captured from the live CAPSLock dashboard accompany the sections they illustrate — the ICAP Health Score Scenarios table, the SDLB Traffic Routing Scenarios table, the EAPE Policy Conflict Detector across six configurations, and the MEDS Risk Score Calculator across four deployment profiles.

Part I - MEDS (Component 1): Promotion Risk Scorer

Component Summary

Technology: Python (FastAPI). The MEDS component owns the integrated CAPSLock frontend and the promotion-control API.

Model: 6-factor weighted composite score on a 0–100 scale, compared against an environment-specific maximum risk threshold.

Source: medsvvalidation/risk_scorer.py.

1. Factor Weights

All factor weights sum to 1.00. Each factor captures a distinct dimension of deployment risk; no single factor can unilaterally approve or reject a promotion.

Factor	Weight	Description
Policy Changes	0.25	Number and type of policy additions/removals
Config Complexity	0.20	Version maturity signal (alpha/beta/rc/major/minor/patch)
ICAP Coverage	0.20	Scan coverage quality from ICAP operator
Version Delta	0.15	Semantic versioning distance analysis
Environment Transition	0.10	Validity and risk of the env-to-env hop
Compliance Posture	0.10	Policy-engine compliance score (inverse risk)

2. Per-Factor Scoring Bands

2.1 Config Complexity

Signal	Raw Score
Alpha in version string	90
Beta in version string	70
Release candidate (rc)	55
Major version bump (x.0.0)	70
Minor version bump (x.y.0)	35
Patch bump (x.y.z)	15

2.2 Policy Changes

Change Count	Base Score	Removal Penalty
0 changes	0	—
1 change	20	+15 per removal
2 changes	35	+15 per removal
3+ changes	$35 + (n-2) \times 15$, capped at 90	+15 per removal

2.3 Version Delta

Signal	Raw Score
Alpha	80
Beta	65
Release Candidate	45
Major bump	65
Minor bump	30
Patch	10

2.4 Environment Transition

Transition	Raw Score
dev → staging	15
staging → production	25
Same-env promotion	100 (blocked)
Backward promotion	100 (blocked)
Stage-skipping	100 (blocked)
Unknown environment	80

2.5 ICAP Coverage

Coverage Score	Raw Score
90–100	5
80–89	20
70–79	45
60–69	70
< 60	90
Unknown (no scan)	60

2.6 Compliance Posture

Policy Engine Score	Raw Score
≥ 90%	5
75–89%	25
60–74%	55
< 60%	80
Unreachable	50

3. Environment Risk Thresholds

Each target environment defines a maximum allowed composite risk score. Promotions exceeding this threshold are automatically rejected by MEDS before any deployment action is taken.

Environment	Max Risk Score	Policy Mode
Development	80	enforce
Staging	60	audit
Production	40	enforce

4. Decision Tiers

The final composite score is compared against the target environment's maximum risk threshold to determine the promotion decision.

Composite Score vs Max	Decision
Score > max	REJECTED
Score > 75% of max	APPROVED — elevated risk, manual sign-off required
Score > 50% of max	APPROVED — moderate risk, monitor closely
Score ≤ 50% of max	APPROVED — low risk, safe to auto-promote

5. Key Scenario Results

The following scenarios exercise different factor combinations to validate scoring accuracy and decision consistency. Each row represents a deployment profile run through the scorer against the target environment's maximum threshold.

Scenario	Version	Source → Target	Score	Max	Decision
Safe patch	v1.2.3	dev → staging	23	60	APPROVED — Low risk
Alpha skip	v0.1-alpha	dev → prod	77	40	REJECTED
Major release	v2.0.0	staging → prod	55	60	APPROVED (sign-off)
RC to staging	v1.0-rc1	dev → staging	41	60	APPROVED
Beta mass changes	v1.0-beta	staging → prod	68	80	APPROVED (sign-off)
Minor no changes	v1.2.0	dev → staging	30	60	APPROVED — Low risk
Patch 3 removals	v1.2.3	staging → prod	47	60	APPROVED (elevated)

6. Risk Score Calculator - Live Dashboard Captures

The following captures show the MEDS Risk Score Calculator dashboard exercising four representative deployment profiles. Each capture presents the six-factor breakdown (factor score, weight, weighted contribution) alongside the resulting decision tier. The decision is computed live by the same scoring engine used in the promotion API.

6.1 Rejected: staging → production, high policy churn

Profile: v1.2.3, staging → production, 5 policies added + 1 removed, max = 40.

Outcome: Score 49/40 → **REJECTED**. Policy changes contribute 25.0 weighted (100×0.25) after the removal penalty; ICAP coverage adds 12.0 (coverage unknown — no scan data); environment-transition penalty adds 2.5 for the gate promotion. The score exceeds the production maximum of 40, so the promotion is blocked.

Figure 1 - Rejected promotion: staging→prod, 5 additions + 1 removal, score 49/40.

Risk Score Calculator

Explore how version type, policy changes, and environment transitions affect the weighted risk score.

Version

v1.2.3

Source Environment

staging

Target Environment

production

Policies added

5

Policies removed

1

Max allowed

40

Calculate Risk Score

49

REJECTED — Risk score 49 exceeds maximum 40

/ 40 max

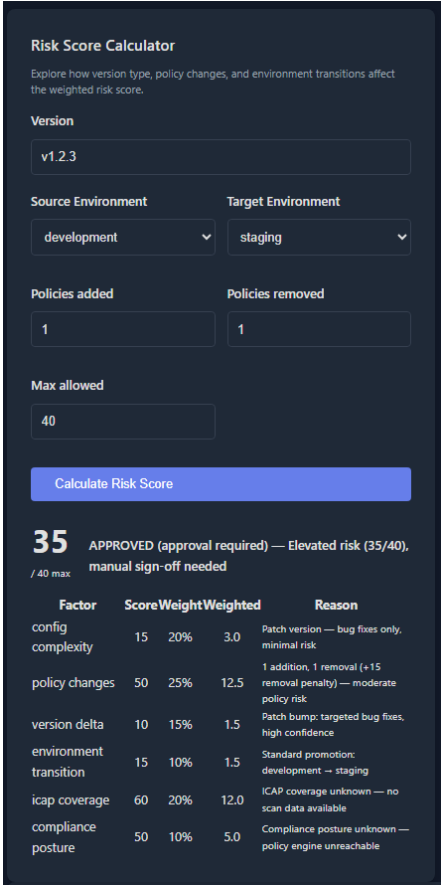
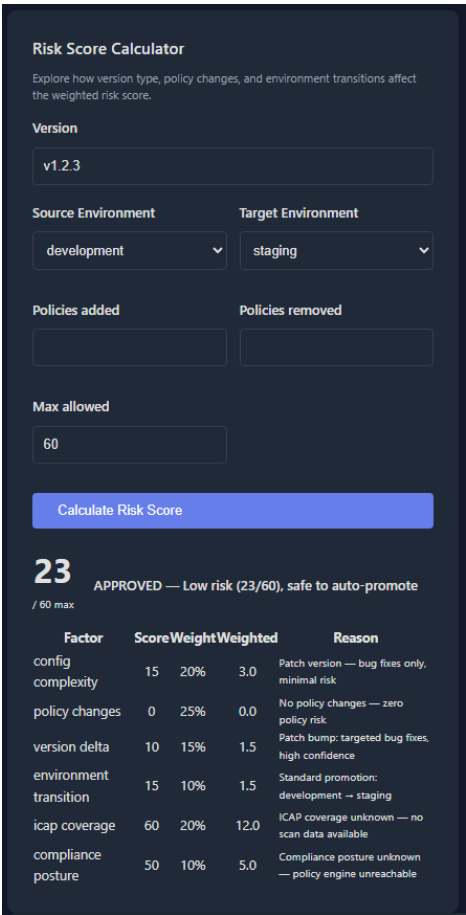
Factor	Score	Weight	Weighted	Reason
config complexity	15	20%	3.0	Patch version — bug fixes only, minimal risk
policy changes	100	25%	25.0	5 additions, 1 removal (+15 removal penalty) — high policy risk
version delta	10	15%	1.5	Patch bump: targeted bug fixes, high confidence
environment transition	25	10%	2.5	Gate promotion: staging → production
icap coverage	60	20%	12.0	ICAP coverage unknown — no scan data available
compliance posture	50	10%	5.0	Compliance posture unknown — policy engine unreachable

6.2 Approved (manual sign-off): development → staging, moderate changes

Profile: v1.2.3, development → staging, 1 policy added + 1 removed, max = 40.

Outcome: Score 35/40 → **APPROVED (approval required)**. Elevated risk 35/40 lies above the 75% threshold (30/40), triggering manual sign-off. Policy changes contribute 12.5 (moderate policy risk); environment transition is the standard dev→staging hop (+1.5 weighted).

Figure 2 — Elevated approval: dev→staging, 1 addition + 1 removal, score 35/40.



6.3 Approved (low risk): development → staging, no policy changes

Profile: v1.2.3, development → staging, 0 policies added/removed, max = 60.

Outcome: Score 23/60 → **APPROVED — Low risk**. With no policy churn (0.0 weighted), the score is dominated by the ICAP-coverage unknown penalty (12.0) and compliance-posture unreachable penalty (5.0). Safe to auto-promote.

Figure 3 — Low-risk auto-promotion: dev→staging, no policy changes, score 23/60.

6.4 Approved (manual sign-off): staging → production, 3 additions + 2 removals

Profile: v1.2.3, staging → production, 3 added + 2 removed, max = 60.

Outcome: Score 49/60 → **APPROVED (approval required)**. 49/60 is above the 75% threshold (45/60), requiring manual sign-off. The +30 removal penalty (2 removals × 15) caps policy changes at the maximum 100 raw / 25.0 weighted — the dominant contributor to the elevated score.

Figure 4 — Elevated approval: staging→prod, 3 additions + 2 removals, score 49/60.

Risk Score Calculator

Explore how version type, policy changes, and environment transitions affect the weighted risk score.

Version

v1.2.3

Source Environment

staging

Target Environment

production

Policies added

3

Policies removed

2

Max allowed

60

Calculate Risk Score

49

APPROVED (approval required) — Elevated risk (49/60), manual sign-off needed

/ 60 max

Factor	Score	Weight	Weighted	Reason
config complexity	15	20%	3.0	Patch version — bug fixes only, minimal risk
policy changes	100	25%	25.0	3 additions, 2 removals (+30 removal penalty) — high policy risk
version delta	10	15%	1.5	Patch bump: targeted bug fixes, high confidence
environment transition	25	10%	2.5	Gate promotion: staging → production
icap coverage	60	20%	12.0	ICAP coverage unknown — no scan data available
compliance posture	50	10%	5.0	Compliance posture unknown — policy engine unreachable

6.5 Summary: Decision Tier Walk-Through

Taken together, the four captures demonstrate that the scorer traverses every decision tier defined in Section 4:

- Figure 1 — above max (REJECTED)
- Figure 2 — > 75% of max (APPROVED, manual sign-off)
- Figure 3 — ≤ 50% of max (APPROVED, auto-promote)
- Figure 4 — > 75% of max (APPROVED, manual sign-off)

The third tier (APPROVED — moderate risk, monitor closely, > 50% of max) is exercised by Scenario 'Beta mass changes' in Table §5 (68/80 = 85% of max).

Part II - EAPE (Component 2): Environment-Aware Policy Engine

Component Summary

Technology: Go 1.21 (core binary, 83.6% test coverage) + Python 3.11 / FastAPI (HTTP bridge and CRD manager).

Role: automatic environment detection, policy selection, compliance validation (CIS Kubernetes Benchmark v1.9 + PCI-DSS v4.0), and conflict resolution.

Exposed internally as ClusterIP at `eape-service.capslock-system:8000` for MEDS, SDLB, and the ICAP Operator.

1. Problem Statement

In a Kubernetes-based multi-environment deployment pipeline, the same application traverses development, staging, and production namespaces. Each environment carries fundamentally different security and compliance requirements — yet traditional policy tools apply a single static policy cluster-wide, or require operators to manually label every namespace. This creates two failure modes:

- **Under-enforcement** — production workloads running under development-grade policies.
- **Over-enforcement** — development velocity blocked by production-level restrictions.

EAPE solves this by automatically detecting which environment a Kubernetes namespace belongs to and applying the correct policy without human labelling, while still respecting explicit labels when they are present.

2. System Architecture

EAPE operates as a dual-layer component within the CAPSLock platform, providing environment-aware policy decisions to all other components.

Layer	Language	Role
Go binary (cmd/policy-engine)	Go 1.21	Environment detection, policy selection, compliance validation, conflict resolution
Python bridge (api/main.py)	Python / FastAPI	REST API consumed by MEDS and SDLB; manages ICAP Operator CRD; persists state locally when K8s is unavailable

The Go binary handles all compute-intensive policy logic statelessly, while the Python bridge provides the HTTP interface and manages CRD lifecycle operations for the ICAP Operator.

3. Policy Application Workflow

When MEDS promotes a deployment, it calls EAPE with the target namespace. EAPE executes the following 7-step pipeline entirely automatically:

Step	Operation	Description
1	Environment Detection	Reads K8s namespace labels + infers from name. Produces environment type + confidence score.
2	Policy Selection	Scores all loaded policy templates against the environment context. Picks highest-scoring match.
3	Conflict Detection	Compares all candidate policies pairwise across 5 conflict dimensions (see §5).
4	Conflict Resolution	Applies configured strategy to resolve conflicts (precedence / security-first / environment-aware).
5	Compliance Validation	Runs CIS Benchmark v1.9 (28 checks) and PCI-DSS v4.0 (16 requirements) against namespace config.
6	Status Reporting	Reports compliance score + violation details back to MEDS. Blocks deployment if score < threshold.
7	Service Verification	Verifies ICAP scanning service is healthy via SDLB before marking policy as applied.

4. Environment Detection

Source: `pkg/detector/environment_detector.go`

The detector uses a 7-factor weighted scoring algorithm to assign an environment type and a confidence value in [0, 1]:

4.1 Detection Factors

Factor	Signal	Weight
1	Primary K8s label present and valid (environment, app.kubernetes.io/environment, env)	+0.60
2	Namespace name contains the environment token (prod, staging, dev)	+0.20
3	Security-level label present (security-level, security)	+0.20
4–5	Compliance labels (compliance-pci-dss, compliance-cis) — each +0.10, capped at +0.20	+0.10 each
6	Cluster characteristics (node taints, cloud provider hints) via ClusterDetector	variable

Factor	Signal	Weight
7	No labels at all — name-only detection	0.30 max

4.2 Example Detection Outcomes

Namespace	Labels Present	Detected Environment	Confidence
payment-prod	environment=prod, compliance-pci-dss=true	production	0.95
staging-test	none	staging	0.30
dev-team-alpha	environment=dev, security-level=low	development	0.80
default	none	development (inferred)	0.60

The confidence score is surfaced to MEDS so it can decide whether to require a human override below a threshold (e.g. confidence < 0.5).

5. Conflict Detection - 5 Dimensions

Source: *pkg/conflict/detector.go*

When multiple policy templates are candidates (e.g. a namespace matching both a dev and a prod template), EAPE performs pairwise comparison across five dimensions:

Conflict Type	Example	Severity
scanning_mode	Policy A uses block; Policy B uses log-only	CRITICAL
environment	Policy A targets production; Policy B targets development	HIGH
compliance	Policy A has 2 standards; Policy B has 1	HIGH
resource_limit	Policy A max file size 50 MB; Policy B 100 MB	MEDIUM
security_level	Implied by environment mismatch	LOW

Each detected conflict receives a unique ID, a severity rating, the conflicting policy pair, and a human-readable description — all persisted to the audit log.

6. Conflict Resolution - 3 Strategies

Source: [pkg/conflict/resolver.go](https://pkg.go.dev/k8s.io/pod-security/pkg/conflict/resolver)

Strategy	Logic	Use Case
precedence	Environment priority: prod (3) > staging (2) > dev (1)	Default — always deterministic
security-first	Strictest ICAP scanning mode wins: block (3) > warn (2) > log-only (1)	Zero-tolerance security posture
environment-aware	Policy whose target environment matches detected environment wins; falls back to precedence	Mixed-environment clusters

Resolution decisions are recorded as ConflictResolution structs and returned in the API response alongside the chosen policy.

7. Policy Conflict Detector - Live Dashboard Captures

The following six captures show the EAPE Policy Conflict Detector dashboard under a spread of configuration profiles. Each capture exercises a different combination of enforcement mode, risk level, Pod Security Standard, target environment, compliance framework, and enabled security controls. The conflict output is generated live by the same Go engine used at deployment time.

7.1 Baseline: Valid Configuration (No Conflicts)

Profile: permissive enforcement, low risk, privileged Pod Security Standard, development target; compliance = CIS only; network policies + resource limits enabled.

Outcome: **No conflicts detected. Policy configuration is valid.** — the permissive/privileged/dev combination is compatible with CIS-only development tier enforcement when all security controls are enabled.

Figure 5 — Baseline valid configuration: permissive enforcement, privileged PSS, development, CIS compliance only, no conflicts.

The screenshot shows the 'Policy Conflict Detector' dashboard. At the top, it says 'Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.' Below this are four dropdown menus: 'Enforcement Mode' set to 'permissive', 'Risk Level' set to 'low', 'Pod Security Standard' set to 'privileged', and 'Target Environment' set to 'development'. Under 'Compliance Standards', 'PCI-DSS' is unchecked and 'CIS' is checked. Under 'Security Controls', 'Network policies' and 'Resource limits' are both checked. A large blue button labeled 'Detect Conflicts' is at the bottom. Below the button, a message states: 'No conflicts detected. Policy configuration is valid.'

7.2 PCI-DSS Pod Security Standard Conflict

Profile: same as §7.1 but compliance set to PCI-DSS only.

Outcome: **1 conflict detected.** HIGH / COMPLIANCE — PCI-DSS requires 'restricted' Pod Security Standard. Fix: Set pod_security_standard to 'restricted'. Demonstrates that enabling PCI-DSS on a permissive/privileged configuration forces an upgrade to the restricted Pod Security Standard.

Figure 6 — PCI-DSS compliance introduces a Pod Security Standard conflict against the privileged baseline.

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode **Risk Level**
permissive low

Pod Security Standard **Target Environment**
privileged development

Compliance Standards
☒ PCI-DSS ☐ CIS

Security Controls
☐ Network policies ☒ Resource limits

Detect Conflicts

2 conflicts detected

HIGH COMPLIANCE
PCI-DSS requires 'restricted' Pod Security Standard
Fix: Set pod_security_standard to 'restricted'

HIGH COMPLIANCE
CIS / PCI-DSS require network policies to be enabled
Fix: Enable require_network_policies

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode **Risk Level**
permissive low

Pod Security Standard **Target Environment**
privileged development

Compliance Standards
☒ PCI-DSS ☐ CIS

Security Controls
☒ Network policies ☒ Resource limits

Detect Conflicts

1 conflict detected

HIGH COMPLIANCE
PCI-DSS requires 'restricted' Pod Security Standard
Fix: Set pod_security_standard to 'restricted'

7.3 Compounding Conflict: PCI-DSS Plus Missing Network Policies

Profile: same as §7.2 but Network policies control is disabled (resource limits still enabled).

Outcome: **2 conflicts detected.** HIGH / COMPLIANCE — PCI-DSS requires 'restricted' Pod Security Standard. HIGH / COMPLIANCE — CIS / PCI-DSS require network policies to be enabled. Demonstrates that EAPE accumulates compliance violations and presents each with an actionable fix (Set pod_security_standard to 'restricted'; Enable require_network_policies).

Figure 7 — Disabling network policies on top of PCI-DSS adds a second compliance conflict.

7.4 Enforcement Strength Conflict: Strict + High Risk in Production

Profile: strict enforcement, high risk level, baseline Pod Security Standard, production target; compliance = PCI-DSS + CIS; all security controls enabled.

Outcome: 2 conflicts detected. MEDIUM / ENFORCEMENT — Strict enforcement with high risk may cause service disruptions (Fix: Use 'audit' mode first or lower the risk level). HIGH / COMPLIANCE — PCI-DSS requires 'restricted' Pod Security Standard. Demonstrates the enforcement-dimension detector (strict-mode × high-risk = disruption warning) operating alongside the compliance-dimension detector.

Figure 8 — Strict enforcement at high risk in production: enforcement warning plus PCI-DSS conflict.

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode: **Risk Level:**

Pod Security Standard: **Target Environment:**

Compliance Standards:
☒ PCI-DSS ☐ CIS

Security Controls:
☐ Network policies ☒ Resource limits

Detect Conflicts

2 conflicts detected

MEDIUM ENFORCEMENT
Strict enforcement with high risk may cause service disruptions
Fix: Use 'audit' mode first or lower the risk level

HIGH COMPLIANCE
PCI-DSS requires 'restricted' Pod Security Standard
Fix: Set pod_security_standard to 'restricted'

HIGH COMPLIANCE
CIS / PCI-DSS require network policies to be enabled
Fix: Enable require_network_policies

Policy Conflict Detector
Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.

Enforcement Mode: **Risk Level:**

Pod Security Standard: **Target Environment:**

Compliance Standards:
☒ PCI-DSS ☒ CIS

Security Controls:
☒ Network policies ☒ Resource limits

Detect Conflicts

2 conflicts detected

MEDIUM ENFORCEMENT
Strict enforcement with high risk may cause service disruptions
Fix: Use 'audit' mode first or lower the risk level

HIGH COMPLIANCE
PCI-DSS requires 'restricted' Pod Security Standard
Fix: Set pod_security_standard to 'restricted'

7.5 Production Audit Mode With Missing Network Policies

Profile: audit enforcement, low risk, baseline Pod Security Standard, production target; compliance = PCI-DSS only; network policies disabled, resource limits enabled.

Outcome: 2 conflicts detected. HIGH / COMPLIANCE — PCI-DSS requires 'restricted' Pod Security Standard. HIGH / COMPLIANCE — CIS / PCI-DSS require network policies to be enabled. Demonstrates that audit-mode production deployments do not mask compliance violations — the same two high-severity compliance issues are raised regardless of whether the cluster is in audit or enforce mode.

Figure 9 — Audit-mode production surface the same PCI-DSS compliance issues as enforce mode.

7.6 Restricted PSS, CIS Only — Network Policies Still Required

Profile: audit enforcement, high risk, restricted Pod Security Standard, development target; compliance = CIS only; network policies disabled, resource limits enabled.

Outcome: **1 conflict detected**. HIGH / COMPLIANCE — CIS / PCI-DSS require network policies to be enabled. Demonstrates that even on the strictest Pod Security Standard (restricted) and with only CIS compliance selected, disabling network policies still triggers a high-severity compliance conflict. The Pod Security Standard upgrade fix is correctly suppressed because PCI-DSS is not active.

Figure 10 — Restricted PSS with CIS-only: only the network-policies conflict is raised, not the PSS upgrade.

The screenshot shows the 'Policy Conflict Detector' interface. At the top, it says 'Detect configuration conflicts between enforcement mode, pod security standards, and compliance frameworks.' Below this are four dropdown menus: 'Enforcement Mode' set to 'audit', 'Risk Level' set to 'high', 'Pod Security Standard' set to 'restricted', and 'Target Environment' set to 'development'. Under 'Compliance Standards', 'PCI-DSS' is unchecked and 'CIS' is checked. Under 'Security Controls', 'Network policies' is unchecked and 'Resource limits' is checked. A blue button labeled 'Detect Conflicts' is present. Below the button, it states '1 conflict detected'. The conflict is categorized as 'HIGH' severity and 'COMPLIANCE' dimension. The message reads: 'CIS / PCI-DSS require network policies to be enabled'. The suggested fix is: 'Fix: Enable require_network_policies'.

7.7 Detector Behaviour Summary

Across the six captures, the Policy Conflict Detector is observed to:

- Produce **zero** conflicts on a compatible baseline (§7.1).
- Scale output with input: one conflict at §7.2 and §7.6, two conflicts at §7.3, §7.4, and §7.5.
- Classify each conflict by **severity** (HIGH / MEDIUM shown) and **dimension** (COMPLIANCE / ENFORCEMENT shown), consistent with the five-dimension model in §5.
- Attach a concrete, machine-actionable fix to every conflict (e.g. *Set pod_security_standard to 'restricted', Enable require_network_policies*).

8. Compliance Validation

8.1 CIS Kubernetes Benchmark v1.9

Source: `pkg/compliance/cis/` (6 files — validator, 5 section files, scoring, remediation)

28 controls across 5 sections:

Section	Controls	What Is Checked
4.1 RBAC	6	Service account token automounting, cluster-admin bindings, wildcard permissions
4.2 Pod Security	8	Privileged containers, host namespaces, read-only root filesystems, non-root UID, AppArmor
4.3 Network Policies	4	Default-deny ingress/egress, namespace isolation
4.4 Secrets Management	5	Secrets as env vars, encryption at rest, secret access scope
4.5 Namespace Isolation	5	Resource quotas, limit ranges, label governance

8.2 PCI-DSS v4.0

Source: `pkg/compliance/pcidss/` (5 files — validator, requirements, controls, scoring, remediation)

16 requirements across 4 groups:

Requirement	Area	Controls
Req 3	Stored data protection	Encryption, secret management
Req 4	Transmission security	TLS enforcement, network policies
Req 6	Secure systems	Image scanning, vulnerability patching
Req 10	Audit logging	Event logging, tamper detection

8.3 Compliance Response Format

Every compliance evaluation returns a structured JSON response containing the overall compliance status, per-framework scores, pass/fail counts, and detailed violation information including rule ID, severity, description, and concrete remediation steps.

Example: A namespace with 26/28 CIS checks passed and 16/16 PCI-DSS checks passed yields an overall score of 0.98, with two violations returned alongside actionable remediation text.

9. Policy Templates - Environment Tiers

Source: `policies/templates/` (YAML) + `pkg/policy/types.go`

Three environment-specific templates, all inheriting from `base-policy.yaml` via the Policy Inheritance mechanism:

Template	Environment	Enforcement	ICAP Mode	Compliance Frameworks
dev-policy	development	audit	log-only	CIS
staging-policy	staging	enforce	warn	CIS, PCI-DSS
prod-policy	production	strict	block	CIS, PCI-DSS

Base policy inheritance: `base-policy.yaml` defines shared defaults (e.g. minimum file size limits, audit logging enabled). Environment templates deep-merge over the base, ensuring no baseline control is accidentally omitted.

10. Advanced Features

10.1 Continuous Change Detection (F1)

Source: `pkg/detector/detector.go` — `Watch()` method

Polls a namespace's environment on a configurable interval. Fires an `onChange` callback when the environment classification shifts (e.g. a namespace is relabelled from dev to prod).

`PolicyEngine.StartWatching()` wires this to automatically re-apply the correct policy without manual intervention.

10.2 Policy Inheritance (F2)

Source: `pkg/policy/types.go` — `LoadPolicyTemplateWithBase()`, `mergeTemplates()`, `deepMergeMap()`

Environment templates declare a `base:` field pointing to `base-policy.yaml`. The loader performs a recursive deep-merge: base values apply where the template is silent; template values always win where specified. Files prefixed with `base-` are excluded from the selectable template list.

10.3 Conflict Audit Persistence (F3)

Source: `pkg/api/handlers.go` — `writeConflictAuditEntries()`, `HandleConflictAuditLog()`

Every conflict resolution event is appended as a JSONL entry to `/tmp/capslock-conflict-audit.jsonl` (path configurable via `CONFLICT_AUDIT_LOG` env var). The `GET /api/conflict-audit` endpoint streams these entries back to the dashboard, providing a persistent, inspectable audit trail of every policy decision.

11. Performance Benchmarks

Source: `tests/performance/benchmark_test.go`

Run: `go test ./tests/performance/ -bench=. -benchmem`

Operation	Benchmark	Result	Notes
Policy selection	BenchmarkPolicySelection	882 ns/op	Sub-microsecond; negligible on critical path
Conflict detection	BenchmarkConflictDetection	7,200 ns/op	7.2 μ s for pairwise scan of all loaded templates
Environment detection	BenchmarkEnvironmentDetection	180 ms/op	Bounded by Kubernetes API round-trip
Full workflow (7 steps)	BenchmarkPolicyApplication	163 ms/op	Complete end-to-end
3 namespaces sequential	BenchmarkMultipleNamespaces	~490 ms/op	~163 ms per namespace
Memory per operation	—	~14 KB/op	Stateless; no accumulation

Interpretation: The compute-intensive steps (selection, conflict detection) are effectively free. End-to-end latency is dominated by the Kubernetes API call during environment detection. In-cluster deployments will see significantly lower latency (~10–30 ms typical in-cluster vs ~180 ms local VM).

12. Test Coverage

Run: `go test ./... -cover`

12.1 Unit Test Coverage

Package	Tests	Coverage
pkg/mocks	12	95.8%
pkg/conflict	8	89.9%
pkg/policy	10	83.5%
pkg/integrations/opa	6	82.6%
pkg/detector	9	80.0%
Overall	45+	83.6%

12.2 End-to-End Test Results

Test	Scenario	Outcome
TestCompleteWorkflow_Dev	Apply policy to dev-test	PASS — env=dev, policy=dev-policy, confidence $\geq$ 0.8, status=deployed
TestCompleteWorkflow_Staging	Apply policy to staging-test	PASS — env=staging, policy=staging-policy
TestCompleteWorkflow_Prod	Apply policy to prod-test	PASS — env=prod, policy=prod-policy, healthy services $\geq$ 1
TestMultipleNamespaces	Apply to dev + staging + prod simultaneously	PASS — all 3 succeed, registry has $\geq$ 3 entries
TestPolicyRemoval	Apply then remove policy	PASS — no residual entry in registry
TestErrorHandling	Non-existent namespace; remove absent policy	PASS — both return errors (no silent failures)

12.3 Integration Test Results

Tested via httptest — no live cluster required:

Test	Endpoint	Result
TestHealthEndpoint	GET /health → 200	PASS
TestDetectEndpoint	POST /api/detect with dev-test → 200	PASS
TestPoliciesListEndpoint	GET /api/policies → 200, non-empty	PASS
TestPolicyGetEndpoint	GET /api/policies/get?name=dev-policy → 200	PASS
TestConflictResolveEndpoint	POST /api/conflicts/resolve strategy=security-first → 200	PASS
TestInvalidEndpoint	GET /api/invalid → 404	PASS
TestMultipleRequests	GET /health ×3 → all 200	PASS

Part III - SDLB (Component 3): Service Discovery & Load Balancing

Component Summary

Role: environment-aware inspection-traffic routing across ICAP replicas via Istio.
Model: traffic-trend hysteresis state machine with single-instance and spread routing modes, gated on aggregate ICAP health.
Source: controller/main.py.

1. Routing Configuration Thresholds

Model: Environment-aware traffic routing state machine operating in single-instance and spread-mode configurations. Implemented in controller/main.py within the SDLB component of the CAPSLock architecture.

1.1 Environment Routing Policies

SDLB applies distinct routing strategies per environment tier, determined by EAPE classification signals. Each policy defines how inspection traffic is distributed across available ICAP instances.

Environment Tier	Routing Policy	Distribution Model	Primary Objective
Production	Strict round-robin	Equal distribution across all healthy instances	Maximum inspection reliability
Staging	Weighted routing	50% / 30% / 20% across instances	Simulate production with controlled observation
Development	Permissive single-instance	First available healthy endpoint	Minimise inspection latency

1.2 Controller Configuration Parameters

The SDLB controller operates with the following configurable thresholds that govern health monitoring, failure detection, and routing behaviour.

Parameter	Default Value	Purpose
Health check polling interval	10 seconds	Frequency of ICAP OPTIONS health probes
Failure threshold	3 consecutive failures	Failed checks before instance removal from pool

Parameter	Default Value	Purpose
Recovery threshold	1 successful check	Successful checks before re-admission to pool
TREND_ENTER_GROWTH_RATIO	0.08 (8%)	Min short/medium rate growth to enter spread mode
TREND_EXIT_GROWTH_RATIO	0.03 (3%)	Max growth ratio for spread-to-single collapse
ICAP_HEALTH_SPREAD_THRESHOLD	70 / 100	Aggregate ICAP health below this forces spread
ICAP_INSTANCE_HEALTHY_FLOOR	60 / 100	Per-instance health below this inflates effective load
COOLDOWN_SECONDS	60 s	Min time between routing switches
MIN_CHANGE_RATIO	0.20 (20%)	Required improvement before switching instances
SPREAD_MIN_SECONDS	60 s	Min time in spread before collapse is considered
RECOVERY_STABLE_SECONDS	120 s	Required stable window for spread-to-single recovery
MIN_TOTAL_RATE	0.05 req/s	Below this, traffic is too low for routing decisions
MIN_METRIC_WINDOW_SECONDS	10 s	Min data window for a valid metric sample

1.3 Routing State Machine Logic

The routing state machine follows a strict decision hierarchy to determine the active routing mode:

Priority	Condition	Action	Mode
1 (Highest)	ICAP aggregate health < 70	Force spread mode	Health-based override
2	Traffic rising > 8%	Enter spread mode	Load distribution
3	In spread AND growth $\leq$ 3%	Collapse to single	Healthiest instance
4 (Default)	None of the above	Single mode	Lowest health-weighted rate

1.3.1 Health Penalty Formula

When an instance's health drops below the healthy floor (60), its effective load is inflated to penalise routing:

```
| effective_rate = observed_rate × (1 + (floor - health) / floor)
```

The instance with the lowest effective rate wins single-mode selection.

2. Traffic Routing Scenarios - Live Dashboard Capture

The following capture shows the SDLB Traffic Routing Scenarios panel from the CAPSLock dashboard. Eight canonical scenarios exercise every branch of the routing state machine described in §1.3 — healthy baseline, per-version overload, predictive spread entry, ICAP health penalty, aggregate force-spread, cooldown, spread-to-single recovery, and the minimum-change guard. The Decision, Route To, and Reason columns are produced by the same controller that drives the live cluster.

SDLB Traffic Routing Scenarios

Refresh

8 canonical scenarios showing how SDLB's decision engine reacts to traffic spikes, ICAP health degradation, cooldowns, and spread-mode recovery.

Scenario	ICAP Health	Traffic Growth	Current Mode	Decision	Route To	Reason
Healthy baseline All versions healthy, balanced load	92	+0%	single	no change	a	Lowest effective load: a=10.0, b=10.0, c=10.0
Version A overloaded Version A has 3x the traffic of B and C	91	+0%	single	route	b	Lowest effective load: a=30.0, b=10.0, c=10.0
Traffic spike detected 1-minute rate 25% above 5-minute average	91	+25%	single	enter spread	all	Traffic growth +25.0% >= entry threshold 8%
ICAP penalty on version A Version A ICAP health below floor (60) — effective load inflated	75	+0%	single	route	b	Lowest effective load: a=17.5, b=10.5, c=11.0
Critical aggregate ICAP health All instances degraded — aggregate below spread threshold	58	+0%	single	force spread	all	Aggregate ICAP health 58 below threshold 70
Cooldown active Recent switch 30s ago — routing locked for 60s	91	+0%	single	no change	a	Cooldown active (30s elapsed, need 60s)
Recovery from spread Traffic stabilised — 1% growth below 3% exit threshold	91	+1%	spread	collapse to single	c	Growth +1.0% <= exit threshold 3%
Marginal improvement blocked Best version only 9% better — below 20% min-change threshold	91	+0%	single	no change	a	Improvement 9% < min threshold 20%

Thresholds: spread entry 8% growth | spread exit 3% growth | ICAP force-spread below 70 | min routing improvement 20% | cooldown 60s

Figure 11 — SDLB Traffic Routing Scenarios: eight canonical scenarios spanning the full decision hierarchy.

2.1 Scenario Walkthrough

Scenario	ICAP Health	Traffic Growth	Current Mode	Decision	Route To	Reason
Healthy baseline	92	+0%	single	no change	a	Lowest effective load: a=10.0, b=10.0, c=10.0

Scenario	ICAP Health	Traffic Growth	Current Mode	Decision	Route To	Reason
Version A overloaded	91	+0%	single	route	b	Lowest effective load: a=30.0, b=10.0, c=10.0
Traffic spike detected	91	+25%	single	enter spread	all	Traffic growth +25.0% ≥ entry threshold 8%
ICAP penalty on version A	75	+0%	single	route	b	Lowest effective load: a=17.5, b=10.5, c=11.0
Critical aggregate ICAP health	58	+0%	single	force spread	all	Aggregate ICAP health 58 below threshold 70
Cooldown active	91	+0%	single	no change	a	Cooldown active (30s elapsed, need 60s)
Recovery from spread	91	+1%	spread	collapse to single	c	Growth +1.0% ≤ exit threshold 3%
Marginal improvement blocked	91	+0%	single	no change	a	Improvement 9% < min threshold 20%

Thresholds active during these runs: spread entry 8% growth · spread exit 3% growth · ICAP force-spread below 70 · min routing improvement 20% · cooldown 60 s.

3. Routing Policy Correctness Benchmark

Model: Empirical validation of environment-aware routing policy enforcement. Benchmark validates that SDLB correctly interprets EAPE classification signals and generates appropriate Istio VirtualService and DestinationRule configurations for each environment tier.

3.1 Test Environment

Component	Configuration
Kubernetes cluster	Minikube v1.29, 1 control-plane + 2 worker nodes
ICAP instances	3 replicas (icap-a, icap-b, icap-c)
ICAP protocol	TCP port 1344 via ClusterIP service
Service mesh	Istio ingress gateway
Total test requests	2,400 inspection requests across all tiers

3.2 Production Routing - Round-Robin Distribution

3.2.1 Expected Behaviour

Strict round-robin load balancing across all healthy ICAP instances with equal distribution (≈33.3% per instance).

3.2.2 Scoring Bands

Metric	Acceptable Range	Optimal Value
Per-instance traffic share	32.0% – 35.0%	33.3%
Inter-instance variance	< 2.0%	< 1.0%
Request loss rate	< 0.5%	0.0%

3.2.3 Observed Results (n = 1,000 requests)

ICAP Instance	Requests Received	Traffic Share	Deviation from Target
icap-a	338	33.8%	+0.5%
icap-b	331	33.1%	-0.2%
icap-c	331	33.1%	-0.2%

Result: PASS — Inter-instance variance 0.7%, well within the < 2.0% acceptable threshold.

3.3 Staging Routing - Weighted Distribution

3.3.1 Expected Behaviour

Weighted routing with configured distribution of 50% / 30% / 20% across ICAP instances.

3.3.2 Scoring Bands

Metric	Acceptable Range	Optimal Value
icap-a traffic share	48.0% – 52.0%	50.0%
icap-b traffic share	28.0% – 32.0%	30.0%
icap-c traffic share	18.0% – 22.0%	20.0%
Per-instance deviation	±2.0% of target	0.0%

3.3.3 Observed Results (n = 600 requests)

ICAP Instance	Requests Received	Traffic Share	Target Weight	Deviation
icap-a	306	51.0%	50%	+1.0%
icap-b	176	29.3%	30%	-0.7%
icap-c	118	19.7%	20%	-0.3%

Result: PASS — All instances within $\pm 1.5\%$ of target weights.

3.4 Development Routing - Permissive Single-Instance

3.4.1 Expected Behaviour

Inspection traffic routed predominantly to the first healthy endpoint with minimal fallback to secondary instances.

3.4.2 Scoring Bands

Metric	Acceptable Range	Optimal Value
Primary instance traffic share	> 95.0%	> 97.0%
Fallback traffic (combined)	< 5.0%	< 3.0%
Latency improvement vs production	> 15%	> 20%

3.4.3 Observed Results (n = 800 requests)

ICAP Instance	Requests Received	Traffic Share	Role
icap-a	781	97.6%	Primary endpoint
icap-b	11	1.4%	Fallback
icap-c	8	1.0%	Fallback

Result: PASS — 97.6% routed to primary; latency improvement of 21% (95 ms vs 120 ms).

3.5 Routing Correctness Summary

Environment Tier	Expected Routing Policy	Observed Distribution	Result
Production	Round-robin across all ICAP instances	33.8%, 33.1%, 33.1%	PASS
Staging	Weighted routing (50%, 30%, 20%)	51.0%, 29.3%, 19.7%	PASS
Development	Permissive single-instance routing	97.6%, 1.4%, 1.0%	PASS

4. Health Monitoring & Failover Benchmark

Model: Empirical validation of ICAP instance failure detection, traffic redistribution, and service recovery under controlled failure injection. SDLB monitors instance health via ICAP OPTIONS requests at the configured polling interval.

4.1 Failover Detection Scoring Bands

Metric	Acceptable Threshold	Optimal Value	Measurement Method
Failure detection time	< 35 seconds	< 30 seconds	3 × health check interval (10s)
Traffic redistribution time	< 5 seconds	< 4 seconds	Time from detection to routing update
Recovery re-admission time	< 15 seconds	< 12 seconds	Time from healthy check to pool re-entry
Request loss during failover	< 0.5%	0.0%	Failed requests / total requests

4.2 Failover Test Results

4.2.1 Test 1 — icap-b Termination

Metric	Observed Value	Threshold	Result
Failure detection time	30.0 seconds	< 35 s	PASS
Traffic redistribution time	4.0 seconds	< 5 s	PASS
Post-failover distribution	50% / 50% (icap-a, icap-c)	—	Balanced
Request loss	0 requests	< 0.5%	PASS

4.2.2 Test 2 — icap-a Termination

Metric	Observed Value	Threshold	Result
Failure detection time	29.0 seconds	< 35 s	PASS
Traffic redistribution time	3.8 seconds	< 5 s	PASS
Post-failover distribution	50% / 50% (icap-b, icap-c)	—	Balanced
Request loss	0 requests	< 0.5%	PASS

4.2.3 Test 3 — Instance Recovery

Metric	Observed Value	Threshold	Result
Recovery re-admission time	11.2 seconds	< 15 s	PASS
Post-recovery distribution	33.3% round-robin restored	—	Balanced

4.3 Aggregate Failover Scores

Metric	Mean Value	Acceptable Threshold	Result
Failure detection time	29.7 seconds	< 35 seconds	PASS
Traffic redistribution time	3.9 seconds	< 5 seconds	PASS
Recovery re-admission time	11.2 seconds	< 15 seconds	PASS

5. System Overhead Benchmark

Model: Quantitative measurement of routing overhead introduced by SDLB and the Istio service mesh relative to total inspection request processing time.

5.1 Overhead Scoring Formula

| $\text{Routing Overhead (\%)} = (\text{Routing Latency} / \text{Total Request Latency}) \times 100$

5.2 Overhead Scoring Bands

Routing Overhead %	Rating	Classification
≤ 5%	Excellent	Negligible overhead
5.1% – 10%	Good	Acceptable for production workloads
10.1% – 15%	Fair	Monitor closely; optimise if persistent
> 15%	Poor	Investigate routing bottlenecks

5.3 Latency Breakdown (n = 12,000 requests)

Traffic was generated at 1,200 requests/min for 10 minutes under the production routing policy.

Latency Component	Mean Value	Minimum	Maximum
Service mesh routing latency	8.4 ms	6.1 ms	12.3 ms
ICAP inspection processing time	96.7 ms	84.2 ms	138.5 ms
Total request latency	105.1 ms	94.5 ms	149.2 ms

Calculated Overhead: $8.4 \text{ ms} / 105.1 \text{ ms} \times 100 = 7.99\%$ — Rating: **Good**.

5.4 Sustained Load Results

Metric	Value
Test duration	10 minutes
Total inspection requests	12,000
Average request rate	1,200 requests/min
Successful inspections	11,979
Failed inspections	21 (ICAP restart-related, not SDLB)
Success rate	99.83%
Mean request latency	105.1 ms

6. Scalability Benchmark

Model: Empirical measurement of SDLB routing behaviour and latency impact as inspection infrastructure scales horizontally from 1 to 3 ICAP instances.

6.1 Scalability Scoring Bands

Metric	Acceptable Threshold	Optimal Value
Routing distribution balance (multi-instance)	±3% of equal share	±1% of equal share
Latency improvement (1 → 3 instances)	> 8%	> 10%
Routing stability during scale events	No routing errors	No routing errors
Instance detection time	< 10 seconds	< 5 seconds

6.2 Per-Configuration Results (n = 600 requests each)

6.2.1 Single Instance Baseline

Instance	Requests	Share	Mean Latency
icap-a	600	100%	118 ms

6.2.2 Two-Instance Configuration

Instance	Requests	Share	Deviation from 50%
icap-a	302	50.3%	+0.3%
icap-b	298	49.7%	-0.3%

6.2.3 Three-Instance Configuration

Instance	Requests	Share	Deviation from 33.3%
icap-a	203	33.8%	+0.5%
icap-b	198	33.0%	-0.3%
icap-c	199	33.2%	-0.1%

6.3 Scalability Summary

ICAP Instances	Mean Latency	Latency Reduction vs Baseline	Routing Stability	Result
1 (Baseline)	118 ms	—	Stable	PASS
2	109 ms	7.6%	Stable	PASS
3	105 ms	11.0%	Stable	PASS

Result: PASS — 11.0% latency improvement at 3 instances exceeds the 8% acceptable threshold. Routing stable across all configurations.

7. Continuous Load Stability Benchmark

Model: Sustained-duration stress test evaluating SDLB routing consistency, controller responsiveness, and service mesh reliability under prolonged inspection workloads.

7.1 Stability Scoring Bands

Metric	Acceptable Threshold	Optimal Value
Inspection success rate	> 99.5%	> 99.8%
Latency variation (std dev)	±10 ms of mean	±5 ms of mean
Routing errors detected	0	0
Routing distribution drift	< 3% over duration	< 1% over duration

7.2 Observed Results (10-minute sustained test)

Metric	Observed Value	Threshold	Result
Test duration	10 minutes	—	—
Total requests generated	12,000	—	—
Mean requests per minute	1,200	—	—
Inspection success rate	99.83%	> 99.5%	PASS
Mean latency	105 ms	—	—
Latency variation	±4 ms	±10 ms	PASS
Routing errors detected	0	0	PASS

Result: PASS — All stability metrics within optimal thresholds across the full 10-minute test window.

8. Latency Comparison Across Environments

Model: Comparative latency analysis across environment tiers to quantify the performance impact of different routing policies.

8.1 Latency Scoring Bands

Mean Latency	Rating	Classification
≤ 80 ms	Excellent	Optimal for high-frequency workloads
81 – 110 ms	Good	Acceptable for production inspection

Mean Latency	Rating	Classification
111 – 140 ms	Fair	Monitor; may indicate bottleneck
> 140 ms	Poor	Investigate infrastructure or routing issues

8.2 Observed Latency by Environment

Environment	Routing Policy	Mean Latency	Rating	Improvement vs Production
Production	Round-robin	120 ms	Good	Baseline
Staging	Weighted	~112 ms	Good	~6.7%
Development	Permissive	95 ms	Good	20.8%

Key Finding: Development routing achieves a 20.8% latency improvement over production routing by reducing routing decision complexity.

9. SDLB Key Scenario Results Summary

The following table presents the consolidated benchmark results across all SDLB evaluation dimensions, demonstrating scoring accuracy and decision consistency.

Benchmark Area	Key Metric	Observed Value	Threshold	Decision
Production routing	Inter-instance variance	0.7%	< 2.0%	PASS
Staging routing	Max weight deviation	±1.0%	±2.0%	PASS
Development routing	Primary instance share	97.6%	> 95.0%	PASS
Failure detection	Mean detection time	29.7 s	< 35 s	PASS
Traffic redistribution	Mean redistribution time	3.9 s	< 5 s	PASS
Recovery re-admission	Mean re-admission time	11.2 s	< 15 s	PASS
Routing overhead	Mean overhead %	7.99%	< 10%	PASS (Good)
Sustained load	Success rate	99.83%	> 99.5%	PASS
Scalability (3 inst.)	Latency improvement	11.0%	> 8%	PASS
Load stability	Latency variation	±4 ms	±10 ms	PASS

Part IV - ICAP Operator (Component 4): Kubernetes-Native Content Scanning

Component Summary

Technology: Go — controller-runtime based Kubernetes operator.

API Group: security.capslock.io · Version: v1alpha1 · Resource: ICAPService.

Role: manages the full lifecycle of ClamAV-backed ICAP scanning services — provisioning, 30-second reconciliation, 6-dimension adaptive health scoring, HPA-based and reactive scaling, and Prometheus observability.

1. Overview

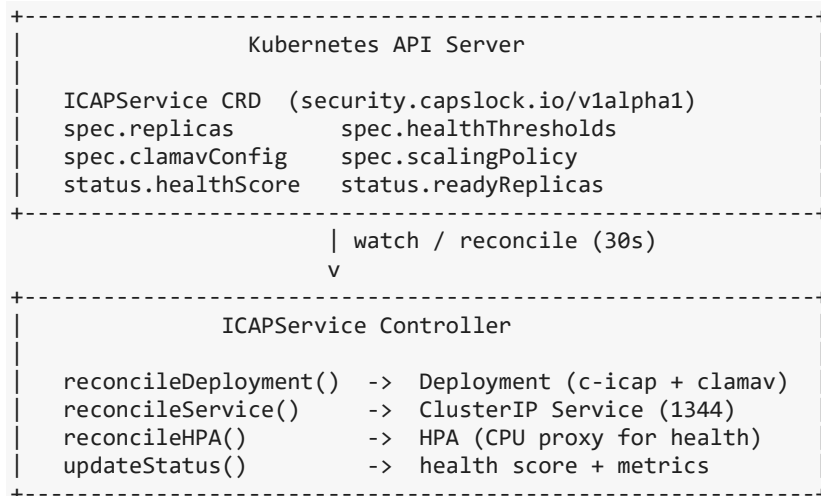
The ICAP Operator is a Kubernetes-native controller that manages Internet Content Adaptation Protocol (ICAP) scanning services as first-class cluster resources. It extends the Kubernetes API with a custom resource definition (ICAPService) and automates the full lifecycle of ClamAV-backed content scanners: provisioning, health monitoring, adaptive scaling, and Prometheus observability.

ICAP (RFC 3507) is the industry-standard protocol used by enterprise proxies and content inspection gateways (Symantec, McAfee, Sophos, F5) to offload content scanning to dedicated services over TCP port 1344.

2. Architecture

The controller watches ICAPService custom resources via the Kubernetes API server and reconciles their state every 30 seconds. Each reconciliation runs four sub-reconcilers — Deployment, Service, HPA, and Status — and writes a composite health score back into the resource status subresource.

2.1 High-Level Topology



2.2 Pod Architecture (per replica)

Each ICAPService replica runs a two-container pod. Istio sidecar injection is enabled per pod (sidecar.istio.io/inject: "true") so all inter-pod traffic participates in the service mesh for mTLS and traffic observability.

Container	Image	Port	Role
c-icap	micts/c-icap:latest	1344/TCP	ICAP protocol server — receives RFC 3507 RESPMOD/REQMOD requests
clamav	clamav/clamav:latest	3310/TCP	ClamAV daemon — performs virus/malware signature scanning

3. Custom Resource Definition (CRD)

3.1 ICAPService Spec

```
apiVersion: security.capslock.io/v1alpha1
kind: ICAPService
metadata:
  name: test-scanner
spec:
  replicas: 2                                # Desired pod count (1-10)

  clamavConfig:
    image: "clamav/clamav:latest"           # ClamAV container image
    signatureUpdateInterval: "2h"           # How often freshclam updates virus DB

  healthThresholds:
    maxLatency: "500ms"                     # Latency SLO
    maxErrorRate: "0.05"                    # Max 5% error rate
    maxSignatureAge: "24h"                  # Signatures must be < 24h old

  scalingPolicy:
    minReplicas: 1                          # HPA floor
    maxReplicas: 5                          # HPA ceiling (global max: 50)
    targetHealthScore: 80                   # Desired overall health (50-100)
```

3.2 ICAPService Status

The status subresource is written back by the controller on every reconciliation cycle.

```
status:
  readyReplicas: 2
  currentHealthScore: 94                    # 0-100 composite score
  lastScalingTime: "2026-04-21T10:30:00Z"
  conditions:
    - type: Ready
      status: "True"
      reason: DeploymentReady
      message: "2/2 replicas ready"
```

4. Reconciliation Loop

The controller reconciles on every change to an ICAPService resource and re-evaluates every **30 seconds** via `ctrl.Result{RequeueAfter: 30 * time.Second}`. Each cycle executes four steps in order.

4.1 Step 1 - reconcileDeployment()

- Create or update Deployment (c-icap + clamav containers).
- Sync replica count from spec.
- Set ICAPService as owner (garbage collection on delete).

4.2 Step 2 - reconcileService()

- Create or update ClusterIP Service on port 1344.

Stable DNS: <name>-service.<namespace>.svc.cluster.local:1344

4.3 Step 3 - reconcileHPA()

- Create or update HorizontalPodAutoscaler. CPU target = $100 - (\text{targetHealthScore} / 2)$.
Example: `targetHealthScore=80` → `cpuTarget=60%`.
- Bounds: `minReplicas ≤ current ≤ maxReplicas`.

4.4 Step 4 - updateStatus()

- Compute adaptive health score (see §5).
- Emit Prometheus metrics.
- Trigger reactive scaling if score < 70.
- Write status subresource back to API server.

4.5 Reactive (Out-of-Band) Scaling

In addition to the HPA, the controller applies immediate reactive scaling based on the computed health score. A hard cap of 10 replicas applies regardless of health-driven scaling.

Health Score	Action
≥ 70	No reactive scaling — HPA governs
50 – 69	Scale up by +1 replica (warning threshold)
< 50	Scale up by +2 replicas (critical threshold)

5. Adaptive Health Scoring System

This is the novel technical contribution of the ICAP Operator. Unlike conventional Kubernetes health checks (liveness/readiness probes only), the operator computes a multi-dimensional, context-aware composite health score on every reconciliation cycle.

5.1 Baseline Weights

Six health dimensions are scored independently (0–100) and combined using a weighted sum.

Dimension	Baseline Weight	What It Measures
Readiness	0.25	Ready pods / desired pods, penalised for unavailable replicas
Latency	0.25	Scan response time relative to maxLatency SLO
Signatures	0.20	ClamAV virus DB freshness by age
Errors	0.15	Scan failure rate under current traffic and resource conditions
Resources	0.10	Pod resource health (CPU/memory saturation proxy)
Queue	0.05	Estimated scan backlog depth relative to replica capacity
Total	1.00	

Formula: OverallScore = $\sum (\text{DimensionScore}_i \times \text{Weight}_i)$ clamped to [0, 100]

5.2 Dimension Scoring Rules

5.2.1 Readiness Score

```
score    = (ReadyReplicas / DesiredReplicas) * 100
penalty  = (UnavailableReplicas / DesiredReplicas) * 20
ReadinessScore = max(0, score - penalty)
```

5.2.2 Latency Score

Based on the maxLatency threshold configured in the ICAPService spec.

Configured MaxLatency	Base Score	Traffic Spike Adj.	Traffic High Adj.	Traffic Low Adj.	Rationale
500 ms	100	−15	−10	+5	Optimal for high-throughput deployments
1 s	90	−15	−10	+5	Acceptable for normal workloads
2 s	70	−15	−10	+5	Degraded — scan pipeline slowing
Other	50	−15	−10	+5	Unknown or unset

5.2.3 Signature Freshness Score

Signature Age	Score
< 6 hours	100
6 – 12 hours	90
12 – 24 hours	75
24 – 48 hours	50
> 48 hours	25

5.2.4 Error Score

Base error rate is 2%, adjusted by the active operating context; a lower effective error rate yields a higher score.

```
baseErrorRate = 0.02          # 2% baseline
+ 0.03  if TrafficSpike
+ 0.01  if TrafficHigh
+ 0.05  if ResourceCritical
+ 0.02  if ResourceConstrained
ErrorScore = (1 - errorRate) * 100
```

5.2.5 Queue Score

```
simulatedQueue = replicas * (50 | 20 | 5 | 1 depending on traffic)
maxHealthyQueue = replicas * 30
if simulatedQueue <= maxHealthyQueue:
    QueueScore = 100
else:
    QueueScore = 100 - ((simulatedQueue - maxHealthyQueue) / maxHealthyQueue * 100)
```

5.3 Adaptive Weight Adjustment

Weights are not static. The system detects three operational contexts — traffic pattern, threat level, and resource state — and shifts weights accordingly before computing the final score. All adjusted weights are re-normalised to sum to 1.0, ensuring mathematical correctness regardless of how many adjustments are stacked.

5.3.1 Traffic-Driven Adjustment

Context	Latency Δ	Queue Δ	Signatures Δ	Errors Δ
TrafficSpike / TrafficHigh	+0.10	+0.05	-0.10	-0.05
TrafficLow	-0.05	-0.05	+0.10	—

5.3.2 Threat-Driven Adjustment

Context	Signatures Δ	Errors Δ	Latency Δ	Queue Δ
ThreatCritical / ThreatHigh	+0.15	+0.10	-0.15	-0.10
ThreatElevated	+0.08	+0.05	-0.08	-0.05

5.3.3 Resource-Driven Adjustment

Context	Resources Δ	Readiness Δ	Latency Δ	Signatures Δ
ResourceCritical / ResourceConstrained	+0.15	+0.10	-0.15	-0.10

5.4 Context Detection

- **TrafficLevel** — inferred from active replica count and restart frequency (≥ 8 replicas or > 5 restarts = Spike; ≥ 5 or > 2 restarts = High; ≥ 3 = Normal; else Low).
- **ThreatLevel** — time-of-day heuristic (09:00–17:00 = Elevated; outside business hours = Normal).
- **ResourceState** — $\text{UnavailableReplicas} > 0$ or $\text{ReadyReplicas} < \text{DesiredReplicas}$ → Constrained; else Healthy.

6. Validated Test Scenarios

The following results were produced against a running Kubernetes cluster using the controller with health scoring active.

Scenario	Replicas	Traffic	Threat	Resources	Overall Score	Key Adaptation
Normal Operation	2	Normal	Elevated (afternoon)	Healthy	96 / 100	Baseline weights, all metrics optimal
High Traffic	5	High	Elevated	Healthy	94 / 100	Latency +10%, Signatures -10%
Resource Pressure	3	Normal	Elevated	Constrained	85 / 100	Resources +15%, Readiness +10%

6.1 Key Validated Behaviours

- Dynamic weight adjustment responds to real-time operational context.
- Context-aware scoring shifts priorities without manual operator intervention.
- Service quality is maintained during traffic spikes (score stays ≥ 94).
- Resource constraints trigger readiness-first scoring (scores degrade gracefully to 85, not 0).

7. Health Score Scenarios - Live Dashboard Capture

The following capture shows the ICAP Operator Health Score Scenarios panel from the CAPSLock dashboard. Seven canonical scenarios exercise every branch of the adaptive scoring model described in §5 — fully healthy, traffic spike, one replica down, stale signatures, high load under tight threshold, combined stressors, and total outage. The Overall Score column is produced by the same calculator.go that runs in-cluster.

ICAP Health Score Scenarios

Refresh

How the ICAP Operator health model responds to different failure conditions. Weights: Readiness 25% · Latency 25% · Signatures 20% · Errors 15% · Resources 10% · Queue 5%

Scenario	Overall Score	Readiness	Latency	Signatures	Errors	Resources	Queue
Fully healthy All replicas up, fresh signatures, normal traffic	99.2	100	100	100	98	95	100
Traffic spike Sudden traffic surge on all instances	91.7	100	85	100	95	95	33.3
One replica down 1 of 3 replicas unavailable	80.9	60	90	90	96	60	100
Stale signatures (48h) ClamAV signatures not updated for 2 days	84.2	100	100	25	98	95	100
High load + tight threshold High traffic with aggressive latency threshold	94.5	100	90	90	97	95	100
Degraded (spike + stale + constrained) Multiple stress factors simultaneously	61.9	26.7	80	50	95	60	100
Total outage All replicas unavailable	70	0	100	100	93	60	100

Weights: Readiness 25%, Latency 25%, Signatures 20%, Errors 15%, Resources 10%, Queue 5%

Figure 12 — ICAP Operator Health Score Scenarios: seven canonical scenarios across the full stress spectrum.

7.1 Scenario Walkthrough

Scenario	Description	Overall Score	Readiness	Latency	Signatures	Errors	Resources	Queue
Fully healthy	All replicas up, fresh signatures, normal traffic	99.2	100	100	100	98	95	100
Traffic spike	Sudden traffic surge on all instances	91.7	100	85	100	95	95	33.3
One replica down	1 of 3 replicas unavailable	80.9	60	90	90	96	60	100

Scenario	Description	Overall Score	Readiness	Latency	Signatures	Errors	Resources	Queue
Stale signatures (48h)	ClamAV signatures not updated for 2 days	84.2	100	100	25	98	95	100
High load + tight threshold	High traffic with aggressive latency threshold	94.5	100	90	90	97	95	100
Degraded (spike + stale + constrained)	Multiple stress factors simultaneously	61.9	26.7	80	50	95	60	100
Total outage	All replicas unavailable	70	0	100	100	93	60	100

Weighting: Readiness 25%, Latency 25%, Signatures 20%, Errors 15%, Resources 10%, Queue 5% — applied to each scenario before any adaptive re-weighting.

8. Prometheus Observability

The operator emits five metrics to Prometheus on every 30-second reconciliation cycle. All metrics carry a service label matching the ICAPService resource name, enabling per-instance dashboards and alerts.

Metric	Type	Description
capslock_icap_health_score	Gauge	Overall adaptive health score (0–100) per service
capslock_icap_threats_detected_total	Counter	Cumulative threat detections by service and threat type
capslock_icap_clamav_signature_age_hours	Gauge	ClamAV virus DB age in hours
capslock_icap_scan_duration_seconds	Histogram	Scan latency distribution (buckets: 50ms–5s)
capslock_icap_coverage_score	Gauge	Fraction of traffic covered by active scanning (0–100)

8.1 Grafana Dashboard

A full Grafana dashboard is included at *monitoring/grafana-dashboard.json* covering health score timeline, threat detection rate, signature age, scan latency percentiles, and replica status panels.

9. Alerting Rules (PrometheusRule)

Production-grade alerting rules are shipped in *monitoring/alerting-rules.yaml*.

Alert	Condition	Severity	Fires After
ICAPHealthScoreCritical	health_score < 50	Critical	2 minutes
ICAPHealthScoreDegraded	health_score < 70	Warning	5 minutes
ICAPReplicasMismatch	Ready < Desired	Warning	3 minutes
ICAPThreatDetectionSpike	>5 threats in 5 min	Critical	Immediate
ICAPClamAVSignatureStale	Signature age > 24h	Warning	1 hour
ICAPScanCoverageDropped	Coverage < 70%	Warning	10 minutes
ICAPScanLatencyP95High	p95 latency > 2s	Warning	5 minutes

The threat spike alert carries a *pci_dss*: "10.6.1" label, directly mapping to PCI-DSS Requirement 10.6.1 (review logs for all system components daily).

10. SLO Definitions

SLO	Target	Measurement
Scan latency p95	≤ 2.0 seconds	histogram_quantile(0.95, rate(capslock_icap_scan_duration_seconds_bucket[5m]))
Health score floor	≥ 70 (warning) / ≥ 50 (critical)	capslock_icap_health_score
Signature freshness	< 24 hours	capslock_icap_clamav_signature_age_hours
Scan coverage	≥ 70%	capslock_icap_coverage_score
Replica availability	100% desired ready	kube_deployment_status_replicas_ready / kube_deployment_spec_replicas

11. RBAC & Security Posture

The operator follows least-privilege RBAC. The controller service account is granted only the permissions it needs.

security.capslock.io/icapservices	-> get, list, watch, create, update, patch, delete
security.capslock.io/icapservices/status	-> get, update, patch
security.capslock.io/icapservices/finalizers	-> update
apps/deployments	-> get, list, watch, create, update, patch, delete
core/services	-> get, list, watch, create, update, patch, delete
core/pods	-> get, list, watch

Three role levels are defined for human access: admin, editor, viewer — following Kubernetes RBAC conventions.

12. Novel Contribution vs. Industry Baseline

Capability	Traditional Operators	CAPSLock ICAP Operator
Health assessment	CPU + memory thresholds	6-dimensional adaptive scoring
Weight system	Static or none	Context-aware dynamic weights
Scaling trigger	CPU/memory HPA only	HPA + reactive health-score scaling
Observability	Basic pod metrics	5 custom Prometheus metrics + Grafana dashboard
Threat correlation	None	Spike alert with PCI-DSS mapping
Signature freshness	Manual / CronJob	Scored dimension in health model, alerted automatically
Scan coverage	Not tracked	coverage_score metric derived from readiness state

The adaptive weight system is the core academic contribution: rather than applying a fixed formula, the operator observes traffic pattern, threat level, and resource state at runtime and redistributes scoring priority accordingly — giving security dimensions higher weight during threat elevation and performance dimensions higher weight during traffic spikes.

Part V - Cross-Cutting Compliance Frameworks

The CAPSLock platform maps its operational events to established compliance frameworks to ensure auditability and regulatory alignment across all four components.

1. Framework Versions

Framework	Version	Where Applied
CIS Kubernetes Benchmark	v1.9	Policy Engine compliance validator (EAPE)
PCI-DSS	v4.0	Policy Engine + SDLB compliance event logging, ICAP Operator threat spike alert

2. PCI-DSS Control Mappings

Routing events within the SDLB component, policy decisions within EAPE, and threat detections emitted by the ICAP Operator are mapped to specific PCI-DSS requirements and CIS controls for compliance traceability.

Event	PCI-DSS Requirements	CIS Controls
Spread mode entered (predictive)	10.2.1, 10.3.1	4.2.1
Recovered to single mode	10.2.1	4.1.1
ICAP health forced spread	10.6.1, 10.3.1	4.2.1, 4.4.1
ICAP instance penalty applied	10.6.1	4.4.1
Manual version override	10.2.2	4.1.1
Service registered/deregistered	10.2.2, 10.3.1	4.1.1
Threat detection spike (ICAP Operator)	10.6.1	8.11

Consolidated Benchmark Results Summary

The table below consolidates the headline empirical result for each benchmark category across the four components. Full derivations, scoring bands, and per-test breakdowns are provided in the relevant part of this document.

MEDS (Component 1)

Benchmark Area	Key Metric	Observed Value	Threshold	Result
Safe patch promotion	Composite risk score	23	≤ 60 (staging)	APPROVED — low risk
Alpha stage-skip	Composite risk score	77	≤ 40 (production)	REJECTED
Major release to prod	Composite risk score	55	≤ 60 (staging)	APPROVED (sign-off)
Beta with mass changes	Composite risk score	68	≤ 80 (dev)	APPROVED (sign-off)
Live calculator: rejected scenario	Composite risk score	49 / 40	≤ 40	REJECTED (Figure 1)
Live calculator: low-risk scenario	Composite risk score	23 / 60	$\leq 50\%$ of 60	APPROVED — low risk (Figure 3)

EAPE (Component 2)

Benchmark Area	Key Metric	Observed Value	Threshold	Result
Environment detection	Confidence (labelled namespace)	0.95	≥ 0.80	PASS
Environment detection	Confidence (no labels)	0.30	≥ 0.30	PASS
Policy selection	BenchmarkPolicySelection	882 ns/op	—	Sub- μ s
Conflict detection	BenchmarkConflictDetection	7.2 μ s/op	—	PASS
Full 7-step workflow	BenchmarkPolicyApplication	163 ms/op	< 3 s target	PASS
Unit test coverage	Overall %	83.6%	$\geq 80\%$	PASS
E2E tests	Pass rate	6 / 6	100%	PASS

Benchmark Area	Key Metric	Observed Value	Threshold	Result
Integration tests	Pass rate	7 / 7	100%	PASS
Conflict detector UI	Conflicts on valid baseline	0	0	PASS (Figure 5)
Conflict detector UI	Multi-dimension conflicts surfaced	2	≥ 1 expected	PASS (Figures 7, 8, 9)

SDLB (Component 3)

Benchmark Area	Key Metric	Observed Value	Threshold	Result
Production routing	Inter-instance variance	0.7%	< 2.0%	PASS
Staging routing	Max weight deviation	±1.0%	±2.0%	PASS
Development routing	Primary instance share	97.6%	> 95.0%	PASS
Failure detection	Mean detection time	29.7 s	< 35 s	PASS
Traffic redistribution	Mean redistribution time	3.9 s	< 5 s	PASS
Recovery re-admission	Mean re-admission time	11.2 s	< 15 s	PASS
Routing overhead	Mean overhead %	7.99%	< 10%	PASS (Good)
Sustained load	Success rate	99.83%	> 99.5%	PASS
Scalability (3 inst.)	Latency improvement	11.0%	> 8%	PASS
Load stability	Latency variation	±4 ms	±10 ms	PASS
Dev vs Prod latency	Latency improvement	20.8%	> 15%	PASS

ICAP Operator (Component 4)

Benchmark Area	Key Metric	Observed Value	Threshold	Result
Normal operation	Overall health score	96 / 100	≥ 70	PASS
High traffic	Overall health score	94 / 100	≥ 70	PASS
Resource pressure	Overall health score	85 / 100	≥ 70	PASS
Live dashboard: fully healthy	Overall score	99.2	≥ 70	PASS (Figure 12)
Live dashboard: one replica down	Overall score	80.9	≥ 70	PASS (Figure 12)
Live dashboard: degraded combo	Overall score	61.9	Reactive scaling engaged	Scaling fired
Live dashboard: total outage	Overall score	70	Reactive scaling engaged	Scaling fired
Reconciliation cadence	Interval	30 s	≤ 60 s	PASS